



UNIVERSIDAD PONTIFICIA DE COMILLAS (ICAI)

DEGREE IN TELECOMMUNICATIONS ENGINEERING

MASTERS THESIS (APPENDIX B)

A Multi-Layer Verification Harness for LLM-Generated Code Patches

Author:

Ignacio Hernández Bas

Supervisor:

Furong Huang

Author's signature:

Supervisor's signature:

12-01-2026

Índice

1	Introduction	2
2	State of the Art	3
3	Motivation	4
4	Objectives	5
5	Methodology	6
5.1	Dataset and Experimental Setup	6
5.2	Verification Pipeline Overview	6
5.3	Static Analysis Layer	7
5.4	Dynamic Verification with Baseline Tests	7
5.5	Behavioral Claim Extraction	7
5.6	Claim Grounding to Patch Modifications	7
5.7	Claim-Based Test Generation	7
5.8	Differential Execution Model	8
5.9	Localized Mutation Testing	8
5.10	Evaluation Protocol and Metrics	9
6	Resources	9
6.1	Datasets	9
6.2	Software Tools and Libraries	9
6.3	Execution and Orchestration Infrastructure	9
6.4	Development Environment	10

1 Introduction

Large Language Models (LLMs) have recently demonstrated strong capabilities in assisting software development tasks, including code generation, bug fixing, and automated program repair [1, 2]. In particular, agent-based systems are now able to generate candidate patches for real-world software issues described in natural language. Despite these advances, ensuring the correctness of AI-generated code remains a fundamental challenge.

In practice, most existing evaluation frameworks assess the correctness of generated patches by executing the test suites provided in the target repository. Benchmarks such as SWE-bench follow this paradigm, considering a patch correct if it passes all available unit tests [3]. However, passing the existing tests does not necessarily imply that the patch fully satisfies the original intent of the issue. Several studies have shown that semantically incorrect or incomplete patches may still pass the test suite, leading to false acceptances and inflated success rates [4].

This limitation arises from the gap between natural-language issue descriptions and executable test coverage. Software issues often specify expected behavior, constraints, or corner cases that are only partially reflected in the existing tests. As a result, purely test-based verification provides weak guarantees of semantic correctness, particularly for logical bugs that do not manifest under the tested scenarios.

At the same time, formally verifying the correctness of arbitrary programs is known to be undecidable [5]. Consequently, practical verification systems rely on approximate and layered approaches that combine multiple sources of evidence, such as static analysis, dynamic execution, and heuristic reasoning, in order to detect incorrect patches with high probability.

This project follows a layered verification philosophy and focuses on strengthening the semantic verification component. Instead of inferring program semantics solely from execution traces or low-level program properties, the approach leverages the natural-language information already present in issue descriptions. Behavioral claims are extracted from the issue text using Natural Language Processing (NLP) techniques and transformed into structured, testable statements that capture the intended behavior of the code. These claims are then grounded to the parts of the program modified by the patch, ensuring relevance and reducing spurious verification signals.

By transforming informal issue descriptions into grounded and testable behavioral claims, the proposed verification framework aims to reduce false acceptances and provide a more faithful assessment of whether an AI-generated patch truly addresses the user’s intent. This document presents the current state of the project, including its motivation, objectives, methodology, and resources.

2 State of the Art

The verification of AI-generated code has received increasing attention as Large Language Models (LLMs) and autonomous agents become increasingly capable of producing patches for real-world software systems. Recent surveys on LLM-based code generation and automated program repair indicate that, while the quality of generated patches has improved substantially, the verification of semantic correctness remains a central open challenge [6, 7]. Existing evaluation and verification approaches can be broadly categorized according to the types of evidence they use to assess correctness: test-based verification, static analysis, differential testing, execution-driven verification, and agentic repair pipelines.

One of the most widely adopted benchmarks for evaluating code-generation systems is SWE-bench [3]. In this benchmark, a model is provided with a natural-language issue description and is evaluated based on whether the generated patch passes the repository’s existing unit tests. The benchmark distinguishes between tests that fail before the fix and should pass afterward (FAIL_TO_PASS) and tests that should continue passing (PASS_TO_PASS). While this methodology enables large-scale and reproducible evaluation, its reliability is fundamentally constrained by the quality and completeness of the existing test suites.

Subsequent studies have shown that SWE-bench-style evaluation can significantly overestimate correctness. In particular, semantically incorrect or incomplete patches may still pass all tests, especially when the test suite does not fully capture the intended behavior described in the issue [4]. To address this limitation, SWE-bench Verified was introduced to improve evaluation reliability by incorporating human-validated tests [8]. However, the verification strategy remains fundamentally test-centric and does not explicitly model the semantic intent or behavioral requirements expressed in the issue description.

Beyond test-based benchmarks, several evaluation harnesses integrate static analysis techniques. The Copilot Evaluation Harness combines dynamic execution with static checks such as syntax validation, linting, and type checking, producing task-specific metrics that go beyond a simple pass/fail outcome [9]. While such techniques improve code quality and maintainability assessment, they provide limited insight into whether the functional behavior of a patch aligns with the expectations described in natural language.

Other frameworks introduce differential or oracle-based verification strategies. PATCHDIFF generates targeted tests by comparing AI-generated patches against developer-provided reference patches, aiming to expose behavioral discrepancies that standard tests may miss [10]. Related efforts in security-focused settings, such as CyberSecEval, employ multi-layer verification pipelines including fuzzing and semantic checks to validate security patches [11]. Although these approaches offer stronger guarantees, they rely on the availability of ground-truth patches or domain-specific assumptions and are therefore unsuitable for general-purpose verification scenarios.

Recent work has explored execution-driven verification and test generation to uncover semantic errors beyond those captured by existing test suites. CodeT demonstrates that automatically generated tests can effectively expose semantic inconsistencies in LLM-generated programs by lever-

aging execution feedback [12]. Similarly, LEVER combines execution signals with learned verification heuristics to assess behavioral correctness beyond static or test-only criteria [13]. While these approaches strengthen verification through execution, they do not explicitly leverage the natural-language intent expressed in issue descriptions.

Agentic code-repair pipelines represent another line of research, in which LLM-based agents iteratively generate, test, and refine patches based on execution feedback [14]. These systems primarily aim to improve patch generation success rates rather than to provide an independent and systematic assessment of semantic correctness. As a result, they typically inherit the same limitations as test-based evaluation, relying on existing tests as the ultimate correctness criterion.

Across these approaches, a common limitation emerges: semantic correctness is assessed indirectly, either through test outcomes, static heuristics, execution signals, or comparisons with reference implementations. The natural-language issue description, which explicitly encodes the user’s intent and expected behavior, is rarely treated as a first-class source of verification evidence. This gap motivates the exploration of verification strategies that directly incorporate semantic information extracted from issue text.

3 Motivation

Despite significant progress in the evaluation of AI-generated code, current verification approaches remain fundamentally limited in their ability to assess semantic correctness. Test-based evaluation assumes that the existing test suite fully captures the intended behavior of the system, an assumption that often does not hold in practice. Static analysis techniques improve code quality and safety but do not guarantee behavioral alignment with user expectations. Differential testing frameworks offer stronger guarantees but depend on the availability of oracle patches, which are rarely accessible in real-world deployment scenarios.

A key observation motivating this work is that software issues are primarily specified in natural language. Issue descriptions frequently contain explicit or implicit statements about expected behavior, constraints, and edge cases that are not exhaustively encoded in executable tests. As a result, the semantic intent of the user is often present in textual form but remains unused during verification. Ignoring this information leads to situations in which patches that technically pass all tests nonetheless violate the original requirements of the issue, a limitation that persists even in benchmarks augmented with human-validated tests [4, 8].

At the same time, prior attempts to infer semantic correctness solely from program executions, execution traces, or inferred invariants face practical challenges. Such approaches may struggle to distinguish between correct and incorrect implementations for logical bugs, particularly when execution coverage is limited or when inferred properties are too weak to characterize the intended behavior. This observation is closely related to prior work on specification mining, where inferred properties are treated as weak specifications derived from partial evidence rather than formal correctness guarantees [15]. More generally, Rice’s theorem implies that no general procedure can decide non-trivial semantic properties of programs, reinforcing the need for approximate and

probabilistic verification strategies [16].

Recent advances in Natural Language Processing provide an opportunity to bridge this gap. Modern NLP models for source code and natural language jointly learn representations that enable structured information to be extracted from informal textual descriptions [17, 18]. These advances make it feasible to treat issue descriptions as weak specifications and to extract behavioral statements that can be operationalized as verification targets.

This project is motivated by the hypothesis that explicitly modeling user intent improves patch verification. Rather than attempting to infer semantics indirectly from executions or low-level program properties, the proposed approach extracts behavioral claims from issue descriptions and represents them as testable assertions about program behavior. To ensure relevance and avoid spurious checks, these claims are grounded to the portions of the code modified by the patch. This grounding step aligns semantic expectations with concrete code changes, reducing noise and focusing verification effort on the intended fix.

By integrating claim extraction and grounding into a layered verification harness, the approach aims to complement existing static and dynamic checks with a semantic layer driven directly by natural-language intent. This design seeks to reduce false acceptances and provide a more faithful assessment of whether an AI-generated patch truly resolves the issue it claims to address.

4 Objectives

The primary objective of this project is to design and implement a multi-layer verification harness capable of evaluating LLM-generated code patches beyond traditional test-based validation.

The specific objectives are as follows:

- Design a modular verification architecture that supports multiple analysis layers.
- Implement a robust dataset ingestion pipeline compatible with SWE-bench Lite and similar benchmarks.
- Develop a static verification layer that analyzes code structure, quality, and potential defects using automated analysis tools.
- Develop a Natural Language Processing (NLP) pipeline to extract behavioral claims from natural-language issue descriptions, representing explicit or strongly implied expectations about program behavior.
- Define a grounding mechanism that links extracted claims to the code elements modified by a patch, ensuring that verification targets are relevant and avoiding spurious semantic checks.

- Generate executable tests from grounded behavioral claims and integrate them into the dynamic verification process.
- Evaluate the effectiveness of claim-based verification in reducing false acceptances compared to baseline test-only evaluation.
- Assess the feasibility, limitations, and practical impact of incorporating natural-language-driven semantic verification into existing code evaluation pipelines.

These objectives are designed to balance theoretical soundness with practical applicability, focusing on techniques that can be deployed in real-world verification settings without requiring ground-truth patches or formal specifications.

5 Methodology

This section describes the methodology used to design, implement, and evaluate the proposed verification framework. The approach follows a layered verification strategy that combines static analysis, dynamic execution, and semantic verification driven by natural-language issue descriptions.

5.1 Dataset and Experimental Setup

The experiments are conducted using a subset of the SWE-bench Lite dataset, which consists of real-world GitHub issues paired with corresponding code patches and test suites. SWE-bench Lite is selected due to its manageable size and its representativeness of common bug-fixing scenarios in widely used Python repositories.

To ensure feasibility and reproducibility, issues are filtered based on repository size, test execution cost, and the observability of expected behavior (e.g., return values or exceptions). All experiments are executed in isolated containerized environments to guarantee consistent dependency resolution and execution conditions across runs.

5.2 Verification Pipeline Overview

The verification pipeline is structured as a sequence of complementary stages. First, static analysis is applied to assess basic code quality and detect structural issues. Second, the repository’s existing test suite is executed to establish baseline functional correctness. Finally, a semantic verification stage evaluates whether the patch satisfies behavioral expectations extracted from the issue description.

Each stage produces independent evidence, and a patch is only accepted if it passes all applicable verification layers. This design allows early rejection of low-quality patches while reserving more expensive semantic checks for plausible candidates.

5.3 Static Analysis Layer

The static analysis layer evaluates code quality prior to execution using established Python analysis tools. These checks target syntactic correctness, type consistency, code complexity, and common bug or security patterns. The outputs of individual tools are aggregated into a composite score that provides a coarse but informative measure of patch quality.

This layer serves two purposes: filtering out low-quality patches early in the pipeline and complementing dynamic verification with maintainability and safety signals that are not captured by test execution alone.

5.4 Dynamic Verification with Baseline Tests

Following static analysis, the patch is applied to the target repository and the existing unit tests are executed in an isolated environment. This step mirrors the standard evaluation procedure used in SWE-bench-style benchmarks and ensures compatibility with established baselines.

Only patches that pass all baseline tests proceed to the semantic verification stage. This ordering prevents unnecessary semantic analysis for patches that already fail functional correctness checks.

5.5 Behavioral Claim Extraction

Semantic verification is based on behavioral claims extracted from the natural-language issue description. A claim represents a testable statement about expected program behavior, such as return values, raised exceptions, or preserved invariants.

Claims are extracted using Natural Language Processing techniques that identify explicit or strongly implied behavioral requirements in the issue text. Each claim is represented in a structured format that captures the condition under which the behavior is expected and the observable outcome.

5.6 Claim Grounding to Patch Modifications

To ensure relevance, extracted claims are grounded to the code elements modified by the patch. Grounding is performed by analyzing the patch diff to identify modified functions, methods, or classes and checking whether the claim targets at least one of these elements.

Claims that cannot be grounded to the patch are discarded, as they are unlikely to be directly affected by the proposed fix. This step reduces noise and prevents semantic checks from targeting unrelated parts of the codebase.

5.7 Claim-Based Test Generation

For each grounded claim, an executable test is generated to operationalize the expected behavior. These tests are designed to be self-contained and to focus exclusively on the behavior described by the claim. The generated tests are integrated into the dynamic verification process and executed alongside the baseline test suite.

If any claim-based test fails, the patch is rejected, as it indicates a violation of the expected semantic behavior derived from the issue description.

5.8 Differential Execution Model

To validate the semantic correctness of a candidate patch, we implement a differential execution harness that evaluates the generated claim-tests across three distinct system states. This approach ensures that a claim c is not only representative of the fix but also sensitive to the underlying logic.

Let $T(c, S)$ be the result of executing the test generated from claim c on system state S . A claim is considered robust and valid if and only if it satisfies the following conditions:

1. **Bug Detection (C_{bug}):** The test must fail on the original, buggy version of the code. $T(c, C_{bug}) = \text{FAIL}$. This confirms that the claim captures the regression or missing behavior described in the issue.
2. **Patch Acceptance (C_{patch}):** The test must pass on the version of the code where the patch has been applied. $T(c, C_{patch}) = \text{PASS}$. This validates that the patch successfully implements the intended semantic change.
3. **Sensitivity to Logic (C_{mut}):** The test must be sensitive to perturbations in the patch’s logic. $\exists m \in \text{Mut}(C_{patch}) \mid T(c, m) = \text{FAIL}$. If a claim-test passes even when the patch is mutated, it is considered too weak or misaligned with the issue’s intent.

5.9 Localized Mutation Testing

The final layer of our verification harness employs localized mutation testing [19] to assess the strength of the semantic claims. Unlike traditional mutation testing which targets the entire code-base, our engine restricts mutations exclusively to the lines modified by the AI-generated patch.

This localization serves two purposes: it reduces the computational overhead associated with large-scale mutation analysis and filters out “noise” from unrelated parts of the system. The process follows these steps:

- **Identification:** The harness analyzes the *git diff* of the patch to identify target lines and symbols.
- **Perturbation:** Standard mutation operators (e.g., operator replacement, condition inversion, or return value modification) are applied to the grounded code elements.
- **Validation:** We calculate the **Mutation Kill Rate (MKR)** for each claim. A claim that fails to “kill” any mutants is flagged as having low semantic depth.

5.10 Evaluation Protocol and Metrics

The effectiveness of the multi-layer harness is measured by its ability to reduce the **False Acceptance Rate (FAR)** compared to baseline evaluations. We define success as the capacity of the semantic layer to reject patches that pass existing repository tests but fail the grounded behavioral claims.

Additionally, we monitor the **Static Quality Index (SQI)**, an aggregate metric derived from static analysis tools (Flake8, Mypy, Bandit) that provides a baseline of code health before dynamic execution.

6 Resources

This section summarizes the main resources used throughout the project, including datasets, software tools, execution infrastructure, and development environments.

6.1 Datasets

The primary dataset used in this project is SWE-bench Lite [3], which contains real-world GitHub issues paired with corresponding code patches and test suites from widely used Python repositories. This dataset provides a realistic benchmark for evaluating AI-generated bug-fixing patches while remaining computationally tractable for iterative experimentation.

6.2 Software Tools and Libraries

The verification framework is implemented primarily in Python. Static analysis relies on established tools for code quality, type checking, and security analysis, including Flake8, Pylint, Mypy, Bandit, and Radon. Dynamic verification is performed using Pytest and repository-specific test runners.

Natural Language Processing components for claim extraction and claim-based test generation are implemented using transformer-based language models accessed through standard machine learning libraries. Additional utilities are used for text parsing, diff analysis, and structured data handling.

6.3 Execution and Orchestration Infrastructure

All verification stages are executed in isolated containerized environments using Singularity containers. Containerization ensures reproducibility, controlled dependency management, and consistent execution of repository test suites across different systems. This approach is particularly suitable for shared or restricted computing environments where traditional Docker-based workflows may not be available, such as the computing cluster of the University of Maryland.

To facilitate interaction with the verification pipeline, a lightweight web interface is implemented using Streamlit. The Streamlit application serves as an orchestration layer, enabling users to select

benchmark instances, inspect patches and issue descriptions, trigger verification stages, and visualize results in an interactive manner. This interface supports rapid experimentation and qualitative analysis without modifying the underlying pipeline.

6.4 Development Environment

The project is developed using standard software engineering tools, including Git for version control and Linux-based development environments. The overall system is designed in a modular manner, allowing individual verification components to be executed independently and facilitating future extensions of the framework.

References

- [1] Mark Chen et al. “Evaluating Large Language Models Trained on Code”. In: *arXiv preprint arXiv:2107.03374* (2021).
- [2] Emilia Hansson and Oliwer Ellréus. “Code correctness and quality in the era of AI code generation”. MA thesis. KTH Royal Institute of Technology, 2023.
- [3] Carlos E. Jimenez et al. “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” In: *The Twelfth International Conference on Learning Representations*. 2024. URL: <https://openreview.net/forum?id=VTF8yNQM66>.
- [4] You Wang, Michael Pradel, and Zhongxin Liu. “Are “Solved Issues” in SWE-bench Really Solved Correctly? An Empirical Study”. In: *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2025.
- [5] Leslie Burkholder. “The Halting Problem”. In: *ACM SIGACT News* 18.3 (1987), pp. 48–60.
- [6] Juyong Jiang et al. “A Survey on Large Language Models for Code Generation”. In: *ACM Transactions on Software Engineering and Methodology* 34.2 (2024). Also available as arXiv:2406.00515, pp. 1–50.
- [7] Boyang Yang et al. “A Survey of LLM-based Automated Program Repair: Taxonomies, Design Paradigms, and Applications”. In: *arXiv preprint arXiv:2506.23749* (2025).
- [8] OpenAI Preparedness Team. *Introducing SWE-bench Verified*. OpenAI Technical Report. 2024. URL: <https://openai.com/index/introducing-swe-bench-verified/>.
- [9] Anisha Agarwal et al. “Copilot Evaluation Harness: Evaluating LLM-Guided Software Programming”. In: *arXiv preprint arXiv:2402.14261* (2024).
- [10] You Wang, Michael Pradel, and Zhongxin Liu. “PATCHDIFF: Dynamic Patch Verification via Differential Testing”. In: *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2025.
- [11] Manish Bhatt et al. “CyberSecEval 2: A Wide-Ranging Cybersecurity Evaluation Suite for Large Language Models”. In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Includes security patch verification benchmarks. 2024.

REFERENCES

- [12] Bei Chen et al. “CodeT: Code Generation with Generated Tests”. In: *The Eleventh International Conference on Learning Representations (ICLR)*. 2023. URL: <https://openreview.net/forum?id=ktrw68Cmu9c>.
- [13] Ansong Ni et al. “LEVER: Learning to Verify Language-to-Code Generation with Execution”. In: *Proceedings of the 40th International Conference on Machine Learning (ICML)*. Vol. 202. PMLR, 2023, pp. 26106–26128.
- [14] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. “RepairAgent: An Autonomous, LLM-Based Agent for Program Repair”. In: *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. Also available as arXiv:2403.17134. 2024.
- [15] Michael D. Ernst et al. “The Daikon System for Dynamic Detection of Likely Invariants”. In: *Science of Computer Programming* 69.1-3 (2007), pp. 35–45.
- [16] Henry Gordon Rice. “Classes of Recursively Enumerable Sets and Their Decision Problems”. In: *Transactions of the American Mathematical Society* 74.2 (1953), pp. 358–366.
- [17] Zhangyin Feng et al. “CodeBERT: A Pre-Trained Model for Programming and Natural Languages”. In: *Findings of the Association for Computational Linguistics: EMNLP 2020*. 2020, pp. 1536–1547.
- [18] Yue Wang et al. “CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation”. In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2021, pp. 8696–8708.
- [19] Yue Jia and Mark Harman. “An Analysis and Survey of the Development of Mutation Testing”. In: *IEEE Transactions on Software Engineering* 37.5 (2011), pp. 649–678.